EECS581: Project 03 Architecture Version 01

Group 6 - Team Members: Kaitlyn Bedgood, Marie Biernacki, Ximena Bustos, Sabelli Antebi Delmas, Greeshma Kunduri, Lauren Lee, Jaydine Stiles

Program Name: My Media Catalog

Program Synopsis: An offline application allowing users to scan, catalog, and locally manage their physical media collection, including books, vinyls, CDs, DVDs, and video games.

Sprint Artifact Declaration

All subsequent sprint releases will include the following on Group 6's Project 3 GitHub repository:

- *Sprint Requirements & Acceptance Criteria*
  - Document that lists the requirements set for the specified sprint and their corresponding definition of done.
- *Sprint Backlog*
  - Document that lists optional items to complete during the specified sprint. These items are only considered after all of the main Sprint Requirements have been met.
- *Meeting Logs*
  - A living document that details what is discussed at team meetings, what tasks are allocated to each team member, and their corresponding deadlines.
- *Accounting of Person Hours*
  - Spreadsheet that details each team member's hours spent on the project during that specific sprint.
- *Sprint Task Tracker*
  - Spreadsheet dedicated to tracking the specified sprint's requirements, whether they are Not Started, In Progress, Blocked, or Completed
- *Expo Go Link/QR Code*
  - Provided for testing the current iteration of the program.

System Architecture Description

System Overview

This project is an offline-first mobile application designed as a personal mobile database for organizing physical media collections using device-based storage, directly fulfilling the architectural specifications detailed in the Group6 Requirements Stack pdf. The system allows

users to seamlessly catalog and locally manage their physical media, encompassing movies, music, and video games, alongside formats like CDs, DVDs, and VHS tapes. Upon launching, the application immediately loads a dedicated home screen that serves as the starting point for the user. The core architectural framework is built using React Native and Expo in VSCode. This specific technology stack was selected to bypass complex emulator configurations. Instead, developers will use the Expo development server to load live app builds over Wi-Fi directly to their physical phones via the Expo Go app. To maximize user autonomy, the application allows individuals to intuitively navigate from the home screen to their media collection, jump to screens for adding or editing media, and smoothly return to the home screen from any other view without friction.

Presentation and User Interface Layer

The front-end user experience is driven by React Native components structured to provide a responsive, modern interface. The primary visual element is a digital catalog that displays all saved media items, which users can view together in one unified collection or organized by specific media types. Users will interact with manual entry forms to add new items by typing in titles, selecting the specific media type and physical format, and assigning relevant genres or subcategories. A crucial dynamic feature of these entry forms is that they strictly display only the information fields relevant to the initially selected media type. Users can also comprehensively manage their catalog by editing existing media information or deleting items. To prevent accidental data loss, the UI implements a confirmation dialog requiring users to confirm before permanently deleting any media item.

For discoverability, users have access to a global search function to quickly find a specific item by its title. The interface supports robust sorting mechanisms, enabling users to sort their media collection alphabetically by title, chronologically by the date added, or hierarchically by user ratings. Filtering options allow users to narrow down their view by media type, physical format, or genre and subcategory. The available genres displayed within the interface will dynamically adapt based on the currently selected media. Additionally, the presentation layer will include a dark mode UI toggle to switch the application's appearance for easier viewing at night.

Data Storage and Management Layer

Because the application prioritizes privacy and offline accessibility, all data management occurs entirely on the device. The architecture leverages the expo-sqlite library to fulfill the core requirement of creating and connecting a local database to store the physical media collection. Whenever the application is opened, it automatically retrieves and loads the user's previously saved media from this database. The local database schemas are highly detailed; alongside basic item details and release years, they include dedicated structures for saving user-generated 1-to-5 star media ratings, as well as personal reviews and notes. Users can

view, edit, or delete these ratings, reviews, and notes at any time, with all changes persistently saved to the database.

Furthermore, the schema captures format-specific metadata, allowing users to save the directors of movies, the artists or bands of albums, and the developers of video games. This additional information is subsequently retrieved and displayed when viewing an item's details. The database efficiently handles full data modification locally, ensuring that any edits to existing media items are saved to the repository and that deleted items are completely and permanently removed from the database records.

Hardware and API Integration

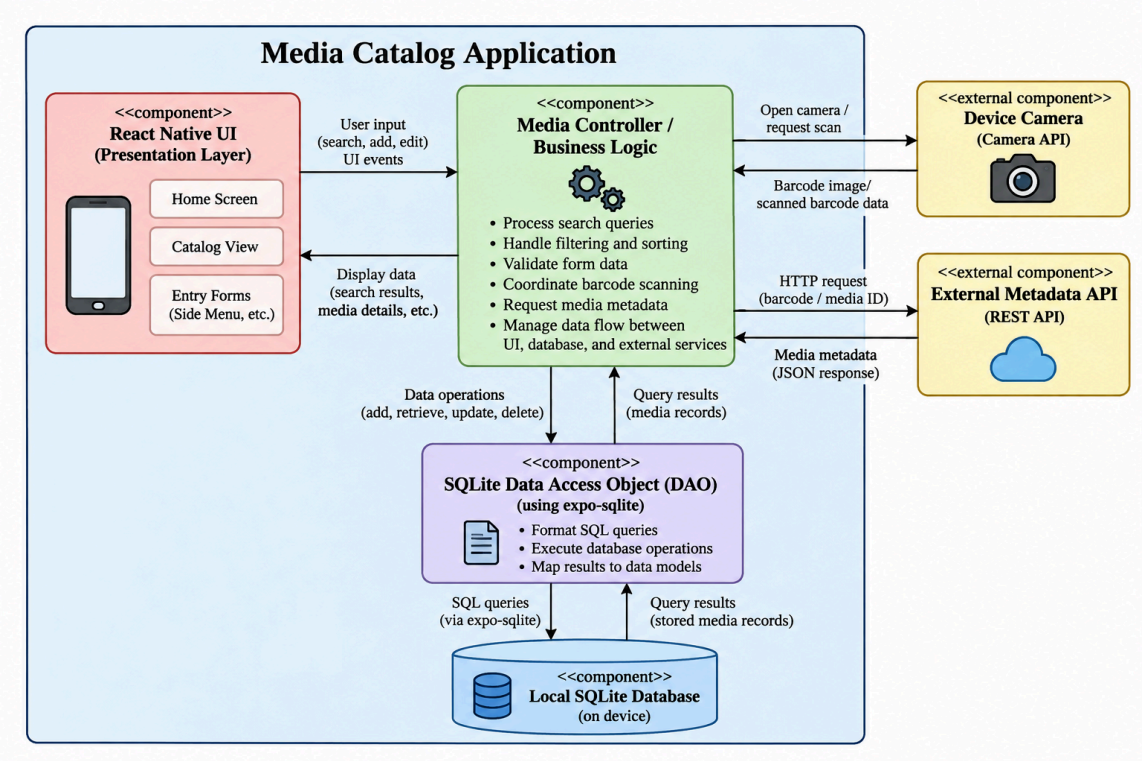
While the app functions primarily offline, it strategically utilizes native device hardware and temporary network requests to streamline data entry. The front end integrates with the device camera, allowing users to scan an item's barcode to quickly identify it. Following a successful scan, the application will temporarily query an external API to fetch barcode metadata and automatically fill in an item's details to save typing before saving the final record to the local database.

UML Diagrams

Component Diagram

This diagram illustrates the high-level structural architecture of the Media Catalog Application, mapping the dependencies between the front-end interface, local storage, and external services.

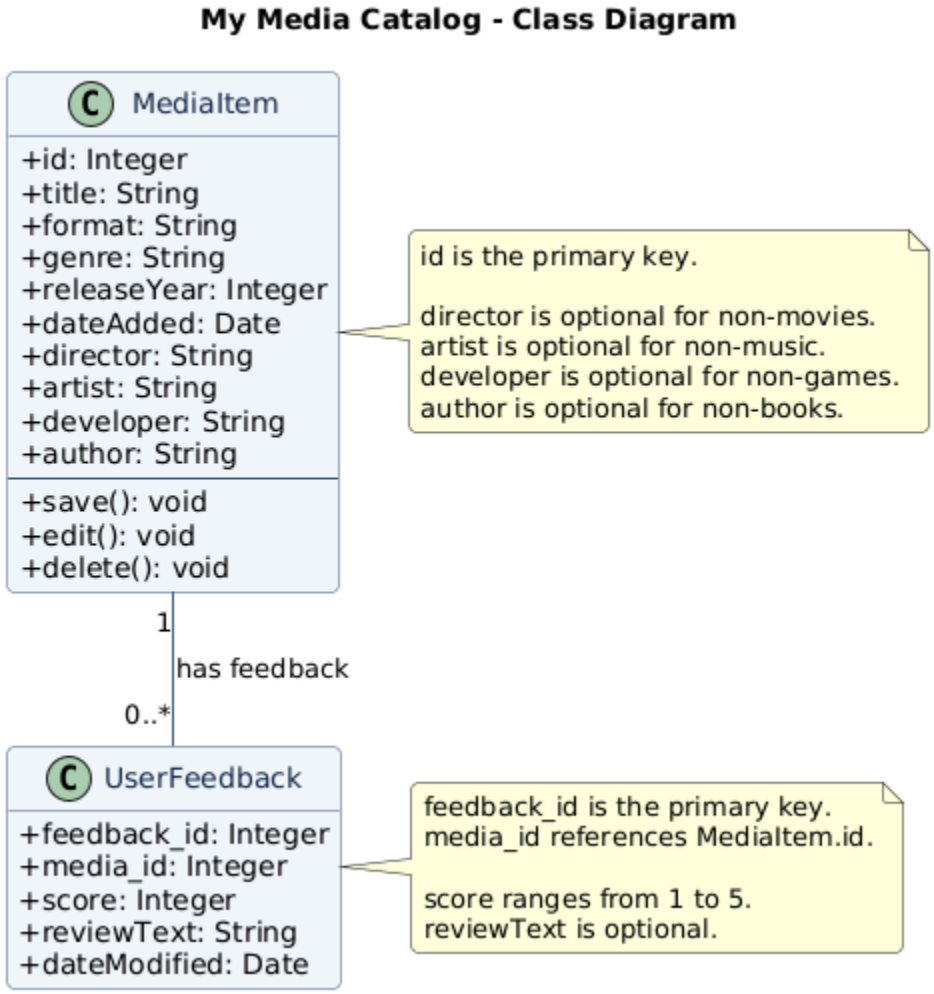
The architecture encapsulates a React Native UI presentation layer that handles user interactions and sends UI events to the Media Controller. This central controller acts as the business logic hub, routing requests to external components outside the system boundary—specifically requesting the device camera for barcode scanning and querying an External API via REST to fetch metadata. For data persistence, the controller interacts with an SQLite Data Access Object (DAO) using the expo-sqlite library, which formats SQL queries and commits the media records to the on-device local SQLite database to ensure offline availability.



Class Diagram

This diagram defines the logical database schema, detailing a Single Table Inheritance setup for media items and a separate associated table for user-generated feedback.

To minimize complex JOIN queries, the core database utilizes a single MediaItem table containing shared attributes like title, format, and dateAdded, alongside format-specific metadata such as director and developer, which are left conditionally nullable depending on the media type. Because user feedback requires a 1-to-many relationship, a separate UserFeedback table is implemented to consolidate 1-to-5 star ratings and text reviews. This associated table connects back to the core MediaItem entity using media_id as a foreign key, ensuring multiple reviews can be reliably attached to a single physical media record.



Sequence Diagram

This diagram maps the behavioral runtime aspect of the application, detailing the chronological flow of messages between software components during a barcode scanning operation.

The sequence initiates when the user triggers the scan via the React Native UI, which requests access to the device camera service. The camera captures the physical barcode and returns the string to the UI, which then executes an asynchronous request to the External Metadata API. The external service returns the formatted media data in JSON, allowing the UI to auto-populate the entry form for the user to review. Upon saving, the finalized data flows to the Database Controller, which inserts the record into the local SQLite database and confirms the successful save back to the UI and the user.

